Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Tecnicatura Universitaria en Inteligencia Artificial

Procesamiento de Imágenes

Trabajo Practico 1

Integrantes del grupo:

Bartomioli, Ornella DNI: 47077671

Villa, Santiago LEGAJO: V-3145/3

Ippolito, Marco LEGAJO: I-0671/8

Problema 1 — Realce de Detalles mediante Ecualización Local

1. Descripción del problema

Se presenta una imagen con un fondo predominantemente blanco que contiene cinco regiones cuadradas negras. El problema principal reside en estas zonas oscuras, donde el **rango dinámico es pobre**, ocultando detalles imperceptibles al ojo humano. El desafío es lograr que la ecualización local se concentre en 'estirar' las intensidades dentro de cada cuadrado negro para revelar los objetos.

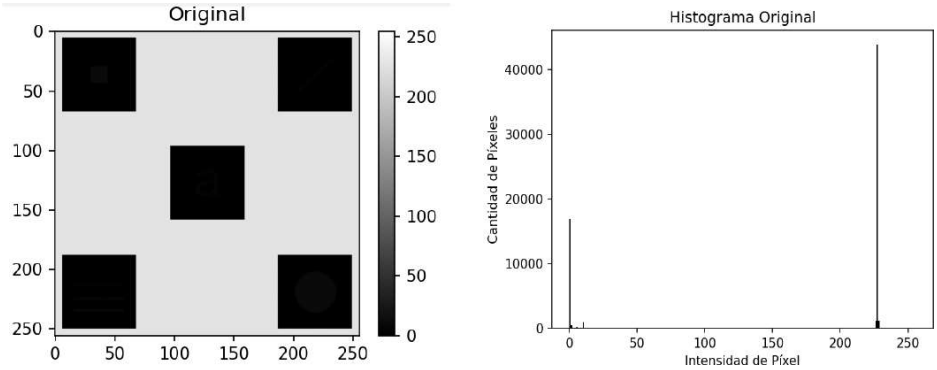


Figura 1.1: imagen original y su histograma

2. Técnicas utilizadas

Para la resolución se optó por la **Ecualización Local de Histograma**. A diferencia de la global, esta técnica calcula la transformación para cada píxel basándose únicamente en su vecindario MxN.

Pasos técnicos implementados:

1. **Lectura y normalización:** se cargó la imagen en escala de grises.
2. **Tratamiento de bordes (padding):** se utilizó cv2.copyMakeBorder con la estrategia BORDER_REPLICATE.
3. **Implementación del barrido:** algoritmo de ventana deslizante con bucles for.
4. **Mapeo de intensidad:** se extrajo cada ROI y se procesó con cv2.equalizeHist.
5. **Análisis de ventanas rectangulares (MxN):** se experimentó con distintas dimensiones.

3. Análisis de problemas enfrentados

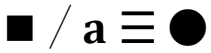
Gestión de Bordes: Uno de los problemas técnicos iniciales fue el manejo de los píxeles periféricos de la imagen. Al intentar centrar la ventana en los bordes, el script arrojaba errores de índice (out of bounds). Esto se resolvió mediante la técnica de Padding, utilizando la función cv2.copyMakeBorder. Se aplicó un borde de tipo BORDER_REPLICATE con un ancho igual al radio de la ventana (M // 2), asegurando así que la iteración fuera fluida en toda la superficie de la matriz sin interrupciones..

Ruido en Ventanas Pequeñas: Al probar con ventanas muy chicas (ej. 7x7), el algoritmo tendía a resaltar el ruido del sensor de la cámara, creando una textura granulada no deseada.

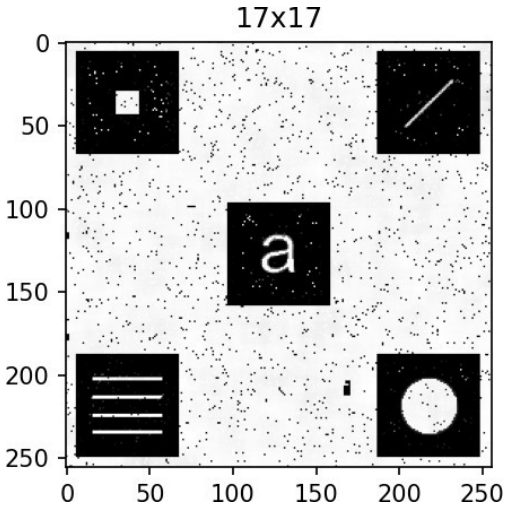
Pérdida de Sensibilidad en Ventanas Grandes: En ventanas grandes (ej. 71x71 o superiores), se enfrentó el problema de la "dilución" del contraste local. Dado que el histograma se calcula sobre un área muy amplia, la estadística de los pequeños objetos oscuros se mezcla con la del fondo blanco predominante. Esto provoca que el algoritmo pierda su capacidad de adaptación local, comportándose de manera similar a una ecualización global y dejando los detalles con un realce insuficiente para su identificación.

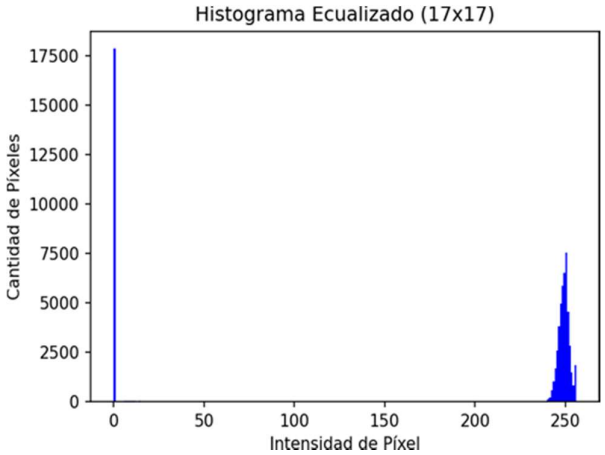
4. Resultados y comparativa

- **Identificación de Objetos:** Gracias al proceso, se lograron identificar los siguientes símbolos**:**



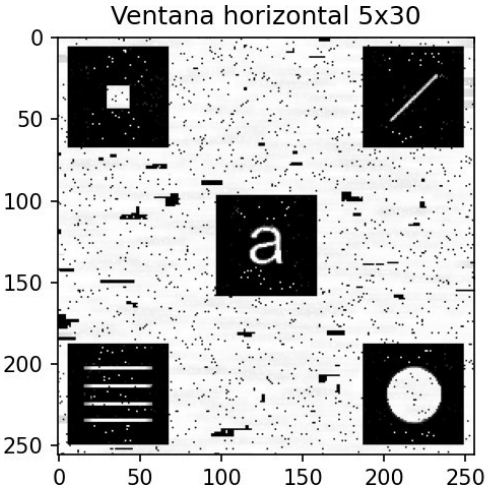
- **Influencia de MxN:** Se determinó que una ventana de 17x17 ofrece el mejor balance. Ventanas más grandes (71x71) suavizan demasiado el efecto, mientras que las más pequeñas (7x7) ensucian la imagen con ruido.

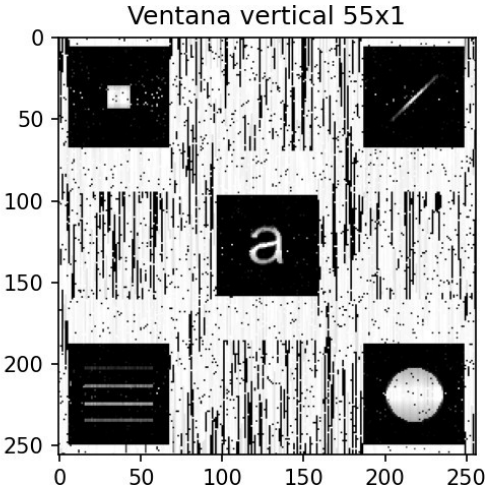




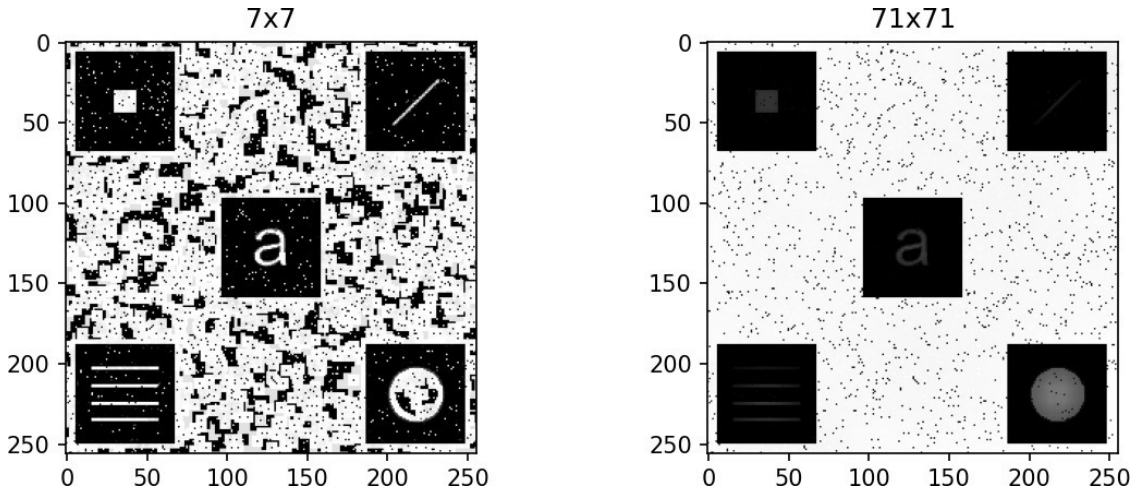
El “sweet spot” haciendo pruebas manuales.

- **Ventanas Rectangulares:** La ecualización se vuelve "direccional". Si usás una ventana muy ancha pero de baja altura (M < N), el algoritmo prioriza el contraste horizontal (y viceversa)





Resultado visual: Los objetos se ven con distorsiones de brillo o "estelas" en la dirección del eje más largo de la ventana. Esto sucede porque el histograma local se construye con píxeles que están lejos en una dirección pero muy cerca en la otra, rompiendo la coherencia espacial del objeto.



Con una ventana chica, la imagen devuelve mucho ruido. Y con una ventana amplia, la imagen no termina de contrastar lo suficiente..

La ecualización local demostró ser superior a cualquier ajuste global para este tipo de imágenes "lavadas". El éxito de la técnica depende críticamente del tamaño del vecindario elegido en relación al tamaño de los objetos que se desean resaltar

Problema 2 — Corrección automática de múltiple choice

Partes A y C — Detección de respuestas y corrección automática

1. Descripción del problema

Se dispone de una serie de imágenes de exámenes de opción múltiple con 10 preguntas cada uno. En cada pregunta el alumno completa una línea en blanco escribiendo a mano la letra de su respuesta (A, B, C o D). El objetivo es desarrollar un script en Python con OpenCV que tome como entrada la imagen de un examen y determine automáticamente cuáles respuestas son correctas y cuáles incorrectas, sin usar las coordenadas de las preguntas como dato de entrada.

2. Descripción del algoritmo

El procesamiento se basa en una **única binarización inicial mediante el método de Otsu**, que calcula automáticamente el umbral óptimo a partir del histograma de cada imagen. Todas las etapas posteriores trabajan sobre la imagen binarizada sin volver a aplicar umbrales. La solución se divide en seis pasos principales aplicados a cada imagen:

Paso 1 — Detección automática de la grilla

La tabla del examen tiene líneas horizontales y verticales bien definidas. Para detectarlas se aplican **operaciones morfológicas**: una erosión con un kernel horizontal alargado deja sobrevivir solo las líneas horizontales (las únicas estructuras suficientemente largas en esa dirección). Lo mismo se hace con un kernel vertical para las líneas verticales. El largo del kernel se define como un porcentaje del tamaño de la imagen (30%), garantizando robustez ante cambios de escala.

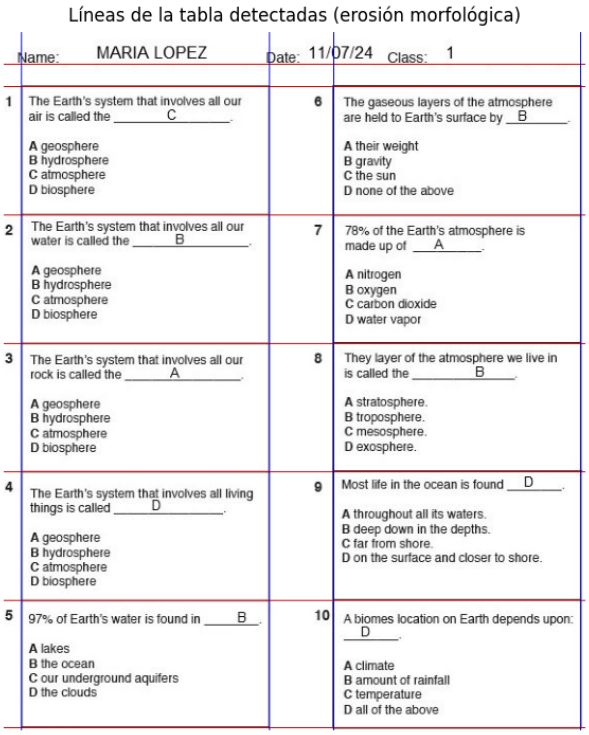


Figura 2.1: líneas de la tabla detectadas mediante erosión morfológica

Paso 2 — Separación del enunciado y las opciones

Dentro de cada celda de pregunta coexisten el enunciado y las opciones A/B/C/D. Entre ellas hay un espacio en blanco que es el más grande de la celda. Se detectan las 'bandas de texto' (grupos de filas con píxeles oscuros) y se identifica el gap máximo. Todo lo que está antes de ese gap es el enunciado; lo que está después son las opciones.

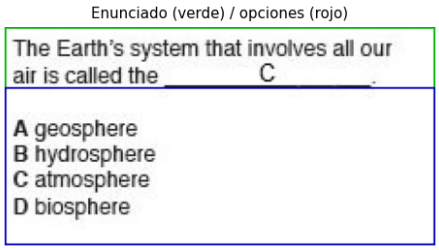


Figura 2.2: enunciado (verde) y opciones (rojo) separadas

Paso 3 — Localización de la línea de guiones y la letra

Dentro de la zona del enunciado se identifican las componentes conectadas mediante cv2.connectedComponentsWithStats. La **línea de guiones** se reconoce como aquella componente cuya altura es menor o igual al 10% del alto de la zona y cuyo ancho supera el 15% del ancho de la zona — es decir, una estructura fina y larga. La **letra escrita** está justo encima: se buscan componentes que solapen horizontalmente con los guiones y cuya base esté a no más de un 20% del alto de la zona por encima del guión.

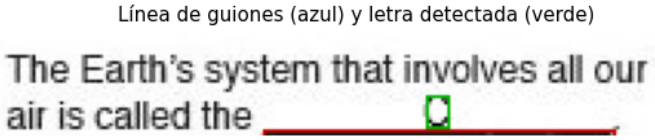


Figura 2.3: línea de guiones (azul) y letra detectada (verde)

Paso 4 — Clasificación de la letra

Se recorta el bounding box de la letra y se clasifica en dos etapas:

1. **Contornos y jerarquía** (cv2.findContours con RETR_TREE): se cuentan los agujeros internos. B tiene 2 agujeros; C no tiene ninguno.
2. **A vs D** (ambas tienen 1 agujero): se compara el ancho máximo de la mitad superior vs la inferior de la letra. La A tiene un travesaño en la mitad baja (ratio inferior/superior mayor a 1.5); la D tiene una curva uniforme (ratio cercano a 1).

Letra	Agujeros	Criterio adicional
B	2	—
C	0	—
A	1	Mitad inferior mucho más ancha (travesaño)
D	1	Mitad superior e inferior similares (curva)

Tabla 2.1: criterios de clasificación de letras

Paso 5 — Detección de múltiple opción y celda vacía

Si hay dos candidatas con área similar (la segunda supera el 60% del área de la primera), la pregunta se considera con múltiple opción marcada e incorrecta. Si no hay candidatas, la pregunta

se considera vacía e incorrecta.

Paso 6 — Comparación con el answer key
La respuesta detectada se compara contra el answer key predefinido. Se imprime OK si coinciden, MAL en caso contrario.

3. Mejoras respecto a la versión inicial

Esta segunda versión incorpora las correcciones señaladas en la devolución oral del trabajo:

Una sola binarización: se eliminaron las múltiples umbralizaciones intermedias. Ahora se aplica un único threshold con Otsu al inicio del procesamiento, y todas las etapas posteriores trabajan sobre la imagen binarizada.

Detección automática de regiones: se eliminaron las coordenadas hardcodeadas. La grilla y los campos del encabezado se detectan automáticamente mediante operaciones morfológicas y análisis de componentes conectadas.

Criterios relativos: todos los umbrales geométricos (alto de líneas, ancho mínimo, márgenes verticales, separación de palabras) se expresan como porcentajes del tamaño de la zona analizada, en lugar de valores absolutos en píxeles. Esto hace al algoritmo robusto frente a deformaciones o cambios de escala de la imagen.

4. Resultados (Parte C)

Examen	Alumno	Correctas	Resultado
examen_1.png	Esteban Alvarez	0/10	DESAPROBADO
examen_2.png	Maria	4/10	DESAPROBADO
examen_3.png	Maria Lopez	10/10	APROBADO
examen_4.png	Lucas Fernandez	0/10	DESAPROBADO
examen_5.png	Juan Perez	10/10	APROBADO

Tabla 2.2: resultados de corrección sobre los 5 exámenes

Partes B y D — Validación del encabezado e imagen de resultados

1. Descripción del problema

Con la misma imagen de entrada se validan los datos del encabezado (Name, Date y Class) según las siguientes restricciones:

- **Name:** debe contener al menos dos palabras y no más de 25 caracteres.
- **Date:** debe tener exactamente 8 caracteres formando una sola palabra.
- **Class:** debe ser un único carácter.

Adicionalmente (Parte D) se genera una imagen de salida con los recortes del campo Name de todos los exámenes, diferenciando visualmente aprobados de desaprobados.

2. Descripción del algoritmo — Parte B

La función **validar_encabezado** trabaja sobre la imagen ya binarizada por la función principal (corregir_examen). El encabezado se identifica automáticamente como la región ubicada arriba de la primera línea horizontal detectada por el algoritmo de detección de grilla — esa primera línea corresponde a las ranuras de relleno donde el alumno escribe Name, Date y Class.

Dentro del encabezado se detectan las **ranuras de los campos** como las líneas horizontales largas situadas en la parte inferior. Cada ranura se identifica por tener un ancho mayor al 5% del ancho del encabezado. Las ranuras se ordenan por posición horizontal para asignarlas en orden a Name, Date y Class.

Para cada campo se recorta la zona de texto justo encima de su ranura y se cuentan los **caracteres como componentes conectadas** mediante cv2.connectedComponentsWithStats. Esta técnica es más robusta que sumar columnas porque permite identificar correctamente caracteres pegados (por ejemplo, los dígitos de una fecha como 11/07/24).

Reglas de validación aplicadas:

- **Name:** se cuentan los espacios entre componentes consecutivas. Un espacio mayor al 3% del ancho de la zona separa palabras. Si hay al menos un espacio de palabras y la cantidad total de caracteres está entre 2 y 25, el campo es válido.
- **Date:** se verifica que la cantidad de componentes sea exactamente 8.
- **Class:** se verifica que haya exactamente 1 componente.

3. Descripción del algoritmo — Parte D

Se genera una figura con matplotlib que muestra, para cada examen, el recorte del campo Name. La zona del campo Name se obtiene automáticamente: se utiliza la primera ranura del encabezado (la más a la izquierda) como límites horizontales del recorte. El recorte se muestra con un borde de color: **verde** si el alumno aprobó (6 o más respuestas correctas) y **rojo** si desaprobó, con la etiqueta correspondiente. Para determinar el resultado se reutiliza la función corregir_examen de las Partes A y C.

4. Mejoras respecto a la versión inicial

Eliminación de coordenadas fijas: en la versión anterior, los campos Name, Date y Class se recortaban con coordenadas absolutas en píxeles. Ahora la posición de cada campo se determina automáticamente a partir de las ranuras de relleno detectadas en el encabezado. Esto permite que el algoritmo funcione correctamente aunque la imagen tenga pequeñas variaciones en el escaneo o el tamaño.

Conteo robusto de caracteres: el conteo basado en grupos de columnas con tinta podía fallar cuando los caracteres estaban pegados (por ejemplo, '11' en una fecha podía contarse como un solo grupo). El uso de componentes conectadas resuelve esto correctamente.

5. Resultados — Parte B

Examen	Alumno	Name	Date	Class
examen_1.png	Esteban Alvarez	MAL	OK	OK
examen_2.png	Maria	MAL	OK	OK
examen_3.png	Maria Lopez	OK	OK	OK
examen_4.png	Lucas Fernandez	OK	MAL	OK
examen_5.png	Juan Perez	OK	OK	OK

Tabla 2.3: resultados de validación del encabezado

El campo Name aparece como MAL en examen_1 (ESTEBANALVAREZ está escrito sin espacio entre nombre y apellido) y en examen_2 (MARIA es una sola palabra). El campo Date aparece como MAL en examen_4 (la fecha escrita no tiene exactamente 8 caracteres). Los demás campos cumplen con las restricciones definidas.

6. Resultados — Parte D

Figura 2.4: imagen de resultados generada por la Parte D

La imagen muestra los recortes del campo Name de los 5 exámenes. Maria Lopez (examen_3) y Juan Perez (examen_5) se muestran con borde verde (APROBADO). El resto con borde rojo (DESAPROBADO).